

PROJECT REPORT

Smart Banking System

A Console-Based Java Application for Banking Account and Transaction Management

Submitted as part of	VITyarthi — Build Your Own Project
Student Name	Nandan Chopra
Registration / Roll No.	25BAI10609
Course / Subject	CSE2006 Programming in Java
Programming Language	Java
Submission Date	17 September 2026

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty members and project guide for their guidance, encouragement and valuable feedback during the development of the Smart Banking System. Their suggestions helped me connect theoretical object-oriented programming concepts with a practical banking application.

I also acknowledge the Java platform documentation and the learning resources consulted while designing the application. Finally, I thank my classmates, friends and family for their support and motivation throughout the completion of this project.

ABSTRACT

The Smart Banking System is a console-based banking application developed in Java. It provides a structured environment for creating and managing customer accounts, authenticating customers through a PIN, performing deposits, withdrawals and transfers, calculating account interest, viewing transaction statements and administering account statuses.

The application supports Savings and Current accounts. It uses inheritance to provide different interest rates while reusing common banking behaviour through an abstract BankAccount class. A HashMap stores accounts and each account maintains an ordered ArrayList of transaction records. Custom checked exceptions handle invalid amounts and insufficient balances. Account data is serialized to accounts.dat so records remain available after the application is restarted.

TABLE OF CONTENTS

1. Introduction
 2. Problem Statement
 3. Objectives
 4. Scope and Users
 5. Requirements Analysis
 6. System Design and Architecture
 7. Object-Oriented Design
 8. Module Description
 9. Implementation Details
 10. Design Diagrams
 11. Testing and Results
 12. Limitations
 13. Future Enhancements
 14. Conclusion
- References

1. INTRODUCTION

Banking operations require accurate account records, controlled access and reliable transaction processing. Even a small banking application must enforce rules such as preventing withdrawals beyond the available balance, recording every successful transaction and restricting operations on closed or frozen accounts.

The Smart Banking System was developed as a lightweight command-line application to model these requirements using core Java. The application provides separate customer and administrator workflows. Customers can manage their own accounts, while administrators can supervise account status and system statistics.

The project is implemented in a single Java source file with multiple related classes. This structure keeps the submission easy to run and inspect while still demonstrating sound object-oriented design. The use of local serialization provides persistence without requiring an external database server.

2. PROBLEM STATEMENT

Manual or unstructured management of customer accounts can lead to incorrect balances, missing transaction records and weak control over account access. A small banking application therefore needs to maintain a central account registry, validate all user inputs, enforce banking rules and preserve records between program sessions.

The project addresses the following problems:

- How can customer accounts be created and identified uniquely?
- How can deposits, withdrawals and transfers be validated consistently?
- How can every successful financial operation be recorded for later review?
- How can frozen and closed accounts be prevented from performing transactions?
- How can an administrator monitor account statuses and overall system statistics?
- How can account data be retained after the application exits?

3. OBJECTIVES

The primary objective is to develop a functional console-based banking application using standard Java and object-oriented programming principles.

The specific objectives are:

- To create Savings and Current accounts with validated customer information.
- To implement PIN-based customer authentication and a failed-login freeze policy.
- To perform deposits, withdrawals and transfers with business-rule validation.
- To maintain transaction history, mini statements and financial summaries.
- To calculate and credit interest according to account type.
- To provide administrator functions for account supervision.
- To demonstrate abstraction, inheritance, polymorphism, interfaces and encapsulation.
- To persist account data using Java object serialization.

4. SCOPE AND USERS

The application is designed for academic demonstration and small-scale local use. It covers account creation, customer authentication, account transactions, transaction reporting, administrative account controls and file-based data persistence.

The system has two categories of users. A **customer** creates an account and uses the customer dashboard to manage transactions and view account information. An **administrator** uses a separate login to search accounts, change account status, inspect statistics and reset daily transaction usage.

The present version does not connect to a real bank, payment gateway, ATM network or external identity provider. It also does not attempt to implement production-grade banking security.

5. REQUIREMENTS ANALYSIS

5.1 Functional Requirements

ID	Requirement
FR-01	The system shall create a Savings or Current account.
FR-02	The system shall generate a unique account number.
FR-03	The system shall validate phone number, email address and four-digit PIN.
FR-04	The system shall authenticate customers using account number and PIN.
FR-05	The system shall freeze an account after three incorrect PIN attempts.
FR-06	The system shall support deposits, withdrawals and account transfers.
FR-07	The system shall reject non-positive, non-finite or limit-exceeding transactions.
FR-08	The system shall reject withdrawals and transfers above the available balance.
FR-09	The system shall record transaction type, amount, description, time and balance after transaction.
FR-10	The system shall display account details, history, mini statements and financial summaries.
FR-11	The administrator shall be able to view, search, freeze, unfreeze and close accounts.
FR-12	The administrator shall be able to view system statistics and reset daily limits.
FR-13	The system shall save and load accounts using a local data file.

5.2 Non-Functional Requirements

Requirement	Implementation in the project
Usability	Numbered menus, simple prompts and repeated validation messages.
Reliability	Custom exceptions and safe handling of invalid saved data.

Performance	Hash-based account lookup and lightweight in-memory collections.
Maintainability	Shared logic is placed in the abstract <code>BankAccount</code> class.
Security baseline	PIN checks, failed-attempt freezing and account-status restrictions.
Persistence	Object serialization through <code>accounts.dat</code> .
Portability	Uses standard Java libraries and no external runtime dependency.

6. SYSTEM DESIGN AND ARCHITECTURE

The system follows a compact layered architecture.

****Presentation layer:**** The `SmartBankingSystem` class displays the main menu, customer dashboard and admin dashboard. It collects input and routes the selected operation.

****Business logic layer:**** The abstract `BankAccount` class validates and executes deposits, withdrawals, transfers and interest operations. It also manages alerts, account status and statements.

****Model layer:**** `SavingsAccount`, `CurrentAccount` and `Transaction` represent the application's core data. Enumerations represent account type, account status and transaction type.

****Persistence layer:**** `saveData()` writes the account map and account counter to `accounts.dat`, while `loadData()` restores them when the program starts.

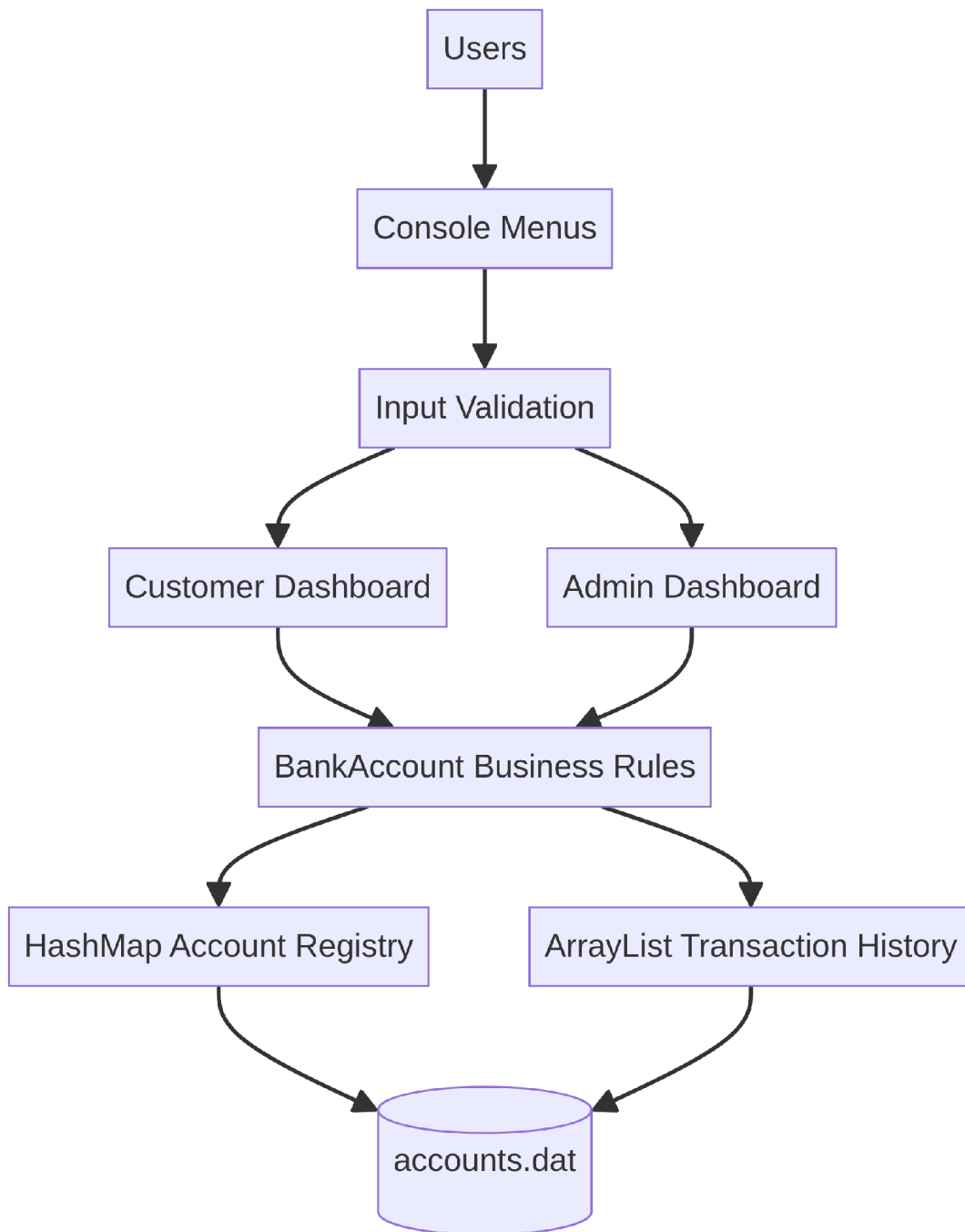


Figure 6.1 — System Architecture

7. OBJECT-ORIENTED DESIGN

7.1 Abstraction

'BankAccount' is declared as an abstract class. It defines common account data and operations while leaving 'getInterestRate()' abstract. The concrete account classes provide their own interest rates.

7.2 Encapsulation

Account fields such as balance, PIN, status and transaction history are private. They are accessed through controlled methods. This prevents direct uncontrolled modification of account state.

7.3 Inheritance

`SavingsAccount` and `CurrentAccount` extend `BankAccount`. They reuse all common banking behaviour and specialize only the account-specific interest rate.

7.4 Polymorphism

The account registry stores objects using the `BankAccount` reference type. At runtime, Java invokes the correct `getInterestRate()` implementation for a Savings or Current account.

7.5 Interface

The `BankingOperations` interface defines `deposit`, `withdraw` and `transfer`. `BankAccount` implements this interface, creating a clear contract for banking operations.

7.6 Exception Handling

`InvalidAmountException` represents invalid or disallowed transaction input. `InsufficientBalanceException` represents an attempt to withdraw or transfer more than the available balance. The menu layer catches these checked exceptions and displays user-friendly messages.

8. MODULE DESCRIPTION

8.1 Account Creation Module

The account creation workflow reads the customer name, phone number, email address, address and PIN. It then asks the user to choose Savings or Current Account. The account counter generates a new account number, and the account is placed in the global `HashMap` before the data is saved.

8.2 Customer Login Module

The customer enters an account number and PIN. Unknown, frozen and closed accounts are rejected. A customer receives three attempts. On the third failure, the account is marked as frozen and saved to the data file.

8.3 Transaction Module

Deposits increase the balance, withdrawals decrease it and transfers update two accounts. A transfer creates one `TRANSFER\_SENT` record for the sender and one `TRANSFER\_RECEIVED` record for the receiver using the same transaction ID. The receiver must be active, and self-transfers are not allowed.

8.4 Interest Module

Savings accounts use a 4.0% interest rate and Current accounts use a 2.0% interest rate. The system first displays estimated interest. If the customer confirms, the interest is added to the balance and recorded as an `INTEREST` transaction.

8.5 Statement Module

The customer can view all transactions, the latest five transactions, account details, daily-limit usage and a financial summary. The summary separately reports deposits, withdrawals, sent transfers, received transfers, interest and current balance.

8.6 Administration Module

The administrator can view all accounts, search a specific account, freeze or unfreeze an account, close an account, view statistics and reset daily limits. An account with a positive balance cannot be closed.

9. IMPLEMENTATION DETAILS

Item	Implementation
Main class	`SmartBankingSystem`
Interface	`BankingOperations`
Abstract class	`BankAccount`
Concrete classes	`SavingsAccount`, `CurrentAccount`
Entity class	`Transaction`
Enumerations	`AccountType`, `AccountStatus`, `TransactionType`
Collections	`HashMap<String, BankAccount>`, `ArrayList<Transaction>`
Account numbering	Starts from `10010001` and increments for each new account.
Transaction IDs	UUID-based values such as `TXN-AB12CD34`.
Low-balance threshold	Rs. 3,000.00
Large-transaction threshold	Rs. 50,000.00
Daily transaction limit	Rs. 100,000.00
Persistence file	`accounts.dat`

9.1 Transaction Validation Algorithm

- Confirm that the account is active.
- Confirm that the amount is greater than zero.
- Confirm that the amount is finite.
- Confirm that the daily transaction limit will not be exceeded.
- For withdrawal or transfer, confirm that sufficient balance exists.
- Update the relevant balance or balances.
- Add the transaction record and display alerts.
- Save the updated account map.

9.2 Transfer Algorithm

The transfer method validates the sender and amount, rejects self-transfer, verifies that the receiver is active, checks the sender's balance, debits the sender, credits the receiver and appends corresponding transaction records. The method uses one transaction ID for both records, which makes the transfer traceable from either account.

10. DESIGN DIAGRAMS

10.1 Use Case Diagram

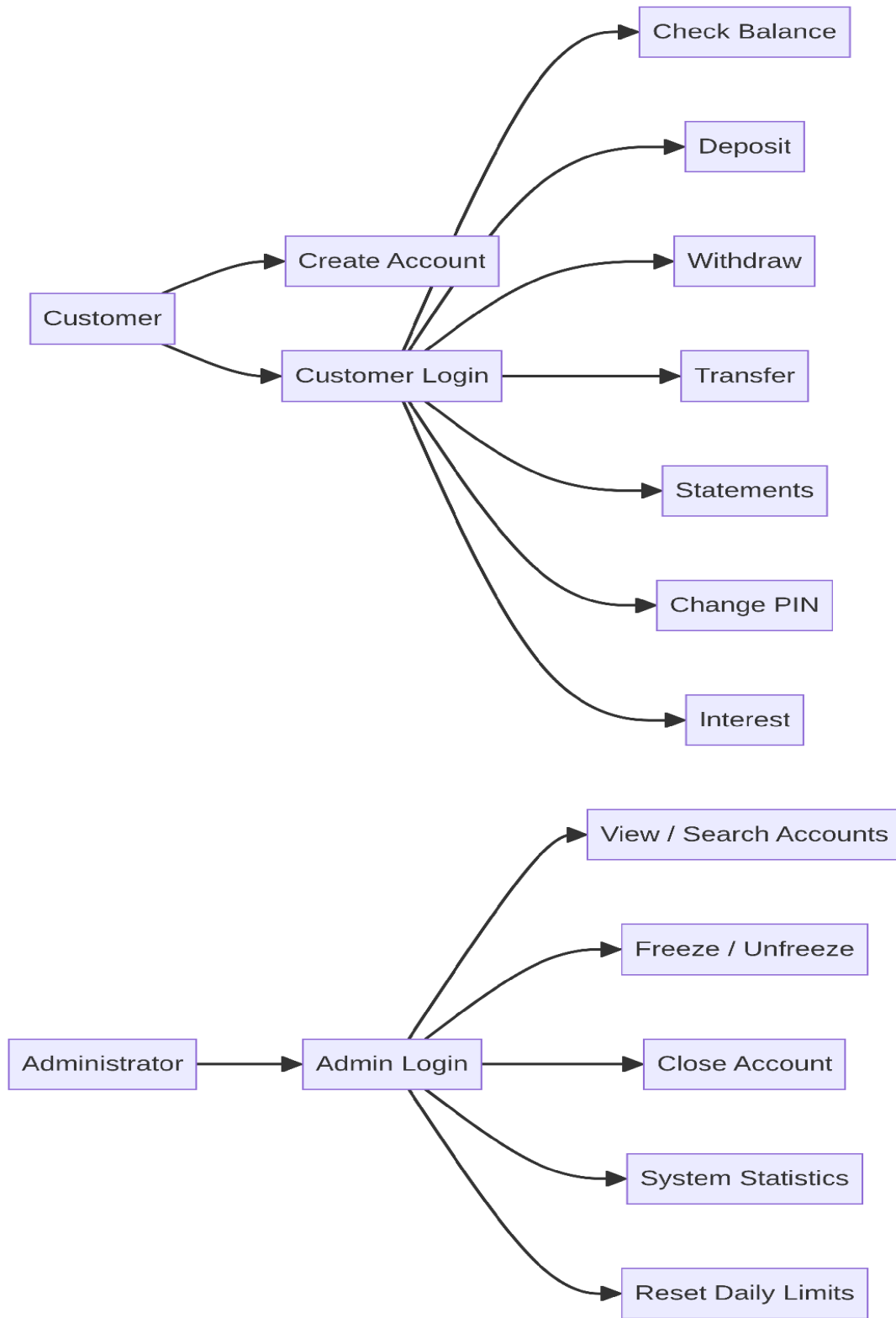


Figure 10.1 — Use Case Diagram

10.2 Transaction Flow Diagram

flowchart TD
S([Start]) --> L[Customer Login]
L --> O[Select Operation]
O --> A[Enter Amount]
A --> V{Valid amount and daily limit?}
V -- No --> E[Display error and retry]
E --> O
V -- Yes --> B{Sufficient balance?}
B -- No --> I[Display insufficient balance]
I --> O
B -- Yes --> U[Update balance]
U --> T[Record transaction]
T --> G[Display alerts]
G --> P[Save data]
P --> F([Finish])

10.3 Class Diagram

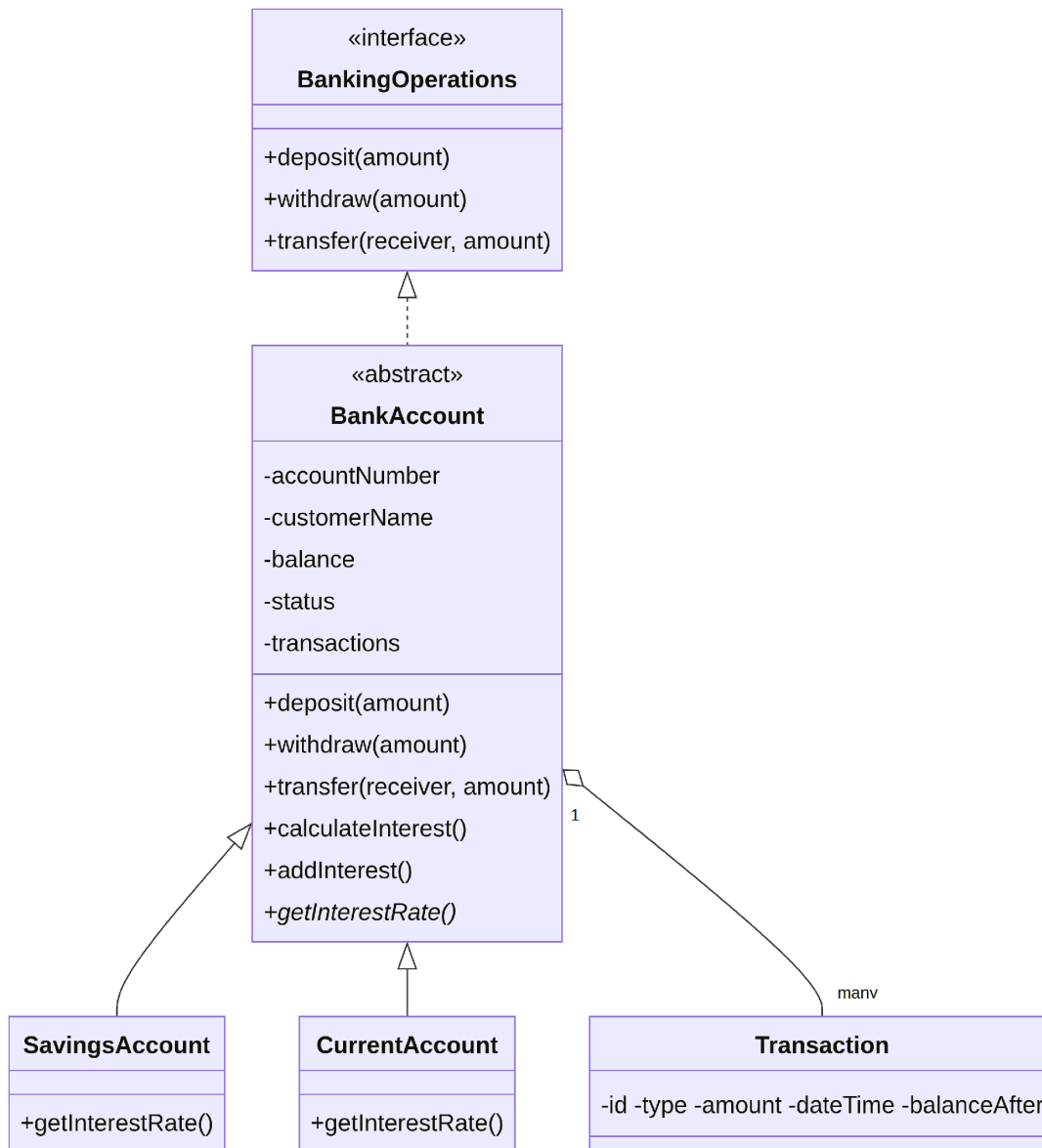


Figure 10.2 — Class Diagram

11. TESTING AND RESULTS

Testing was planned around normal operations, invalid inputs, security conditions and persistence.

Test ID	Test scenario	Expected output
T01	Create account with valid information	Account is created and account number is displayed.

T02	Enter invalid phone number	System asks for a ten-digit phone number again.
T03	Enter invalid email	System rejects the value and repeats the prompt.
T04	Deposit a positive amount	Balance increases and a deposit record is created.
T05	Deposit zero or negative amount	Deposit fails with an invalid-amount message.
T06	Withdraw above balance	Withdrawal fails with an insufficient-balance message.
T07	Transfer to unknown account	Transfer is rejected because receiver is not found.
T08	Transfer to same account	Transfer is rejected.
T09	Three incorrect PIN attempts	Account becomes frozen.
T10	Customer tries to use frozen account	Login and transaction access are denied.
T11	Close account with positive balance	Closure is rejected.
T12	Credit account interest	Correct account-specific interest is added and recorded.
T13	Restart the application	Previously saved accounts and account counter are loaded.
T14	Exceed daily transaction limit	Further applicable transaction is rejected.

Successful operations update the account state and are persisted. Failed operations do not modify the balance or create misleading transaction records. This is particularly important for withdrawals and transfers because the balance must remain unchanged when validation fails.

12. LIMITATIONS

The application is an academic prototype rather than a production banking system. PIN values and administrator credentials are not protected with hashing or encryption. The administrator credentials are hard-coded in the source. Java serialization provides convenient local persistence but does not offer the transactional guarantees, concurrent access control or recovery features of a banking database.

The application uses `double` for monetary values. A production financial system should use `BigDecimal` with explicit rounding rules. Daily-limit usage is reset manually by the administrator rather than automatically when a new calendar day begins. The system also lacks multi-factor authentication, password recovery, audit trails, external notifications and online payment integration.

13. FUTURE ENHANCEMENTS

The next version should use a relational database with repository and service layers. Account and transaction updates should be enclosed in database transactions to guarantee consistency. Monetary values should be migrated to `BigDecimal`.

Security can be improved through salted PIN hashing, secure administrator authentication, role-based access control, multi-factor authentication and audit logging. Additional functionality may include automatic

daily-limit rollover, scheduled interest crediting, statement export to PDF or CSV, email and SMS notifications, customer profile updates, beneficiary management and a graphical or web interface.

Automated unit and integration tests should be added for all business rules. A multi-user version should also introduce synchronization or database-level locking to prevent race conditions during simultaneous transactions.

14. CONCLUSION

The Smart Banking System provides a complete academic demonstration of a small banking application. It supports customer registration, PIN authentication, deposits, withdrawals, transfers, interest crediting, transaction statements, alerts, administrative account controls and persistent storage.

The project effectively connects Java theory with practical application. Abstraction and inheritance are demonstrated through the account hierarchy. Polymorphism is used for account-specific interest rates. The interface defines a common banking contract, collections maintain account and transaction records, custom exceptions communicate domain errors, and serialization preserves data between sessions.

The implementation is suitable for course submission because it has a clear problem definition, defined requirements, modular workflows, explicit business rules, validation logic and test scenarios. Its limitations also identify realistic areas for future improvement, particularly security, financial precision, database persistence and concurrent access.

REFERENCES

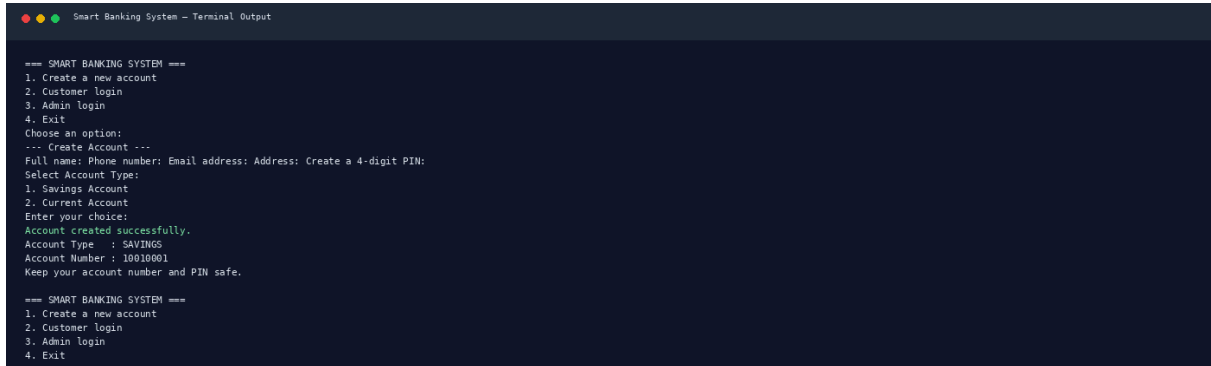
- [1]: <https://docs.oracle.com/en/java/javase/21/docs/api/> "Java Platform, Standard Edition 21 API Documentation"
- [2]: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/Serializable.html> "Java Serializable Interface Documentation"
- [3]: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html> "Java HashMap Class Documentation"
- [4]: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html> "Java ArrayList Class Documentation"

****Project Source Reviewed:**** `SmartBankingSystem.java` ****Report Prepared By:**** [Student Name]
****Academic Session:**** 2026

15. SAMPLE OUTPUT SCREENSHOTS

The following screenshots were captured from an actual execution of SmartBankingSystem.java using Java 21. The run demonstrates account creation, customer login, deposit, withdrawal, mini statement, financial summary, interest estimation and administrator statistics.

15.1 Account Creation and Login



```
Smart Banking System - Terminal Output

=== SMART BANKING SYSTEM ===
1. Create a new account
2. Customer login
3. Admin login
4. Exit
Choose an option:
--- Create Account ---
Full name: Phone number: Email address: Address: Create a 4-digit PIN:
Select Account Type:
1. Savings Account
2. Current Account
Enter your choice:
Account created successfully.
Account Type : SAVINGS
Account Number : 10010001
Keep your account number and PIN safe.

=== SMART BANKING SYSTEM ===
1. Create a new account
2. Customer login
3. Admin login
4. Exit
```

Figure 15.1 — Account creation and customer login output

15.2 Customer Transactions and Statements

```
Smart Banking System - Terminal Output

Choose an option:
--- Customer Login ---
Account number: PIN: Login successful.

--- Customer Dashboard ---
Welcome, Rahul Sharma
Balance: Rs. 0.00
1. Check balance
2. Deposit money
3. Withdraw money
4. Transfer money
5. Transaction history
6. Mini statement
7. Account details
8. Change PIN
9. Financial summary
10. Daily transaction limit
11. Calculate interest
12. Logout
Choose an option: Deposit amount: Rs. Deposit successful. New balance: Rs. 10000.00

--- Customer Dashboard ---
Welcome, Rahul Sharma
Balance: Rs. 10000.00
1. Check balance
2. Deposit money
3. Withdraw money
4. Transfer money
5. Transaction history
6. Mini statement
7. Account details
8. Change PIN
9. Financial summary
10. Daily transaction limit
11. Calculate interest
12. Logout
Choose an option: Withdrawal amount: Rs. Withdrawal successful. Remaining balance: Rs. 8000.00

--- Customer Dashboard ---
Welcome, Rahul Sharma
Balance: Rs. 8000.00
1. Check balance
2. Deposit money
3. Withdraw money
4. Transfer money
5. Transaction history
6. Mini statement
7. Account details
8. Change PIN
9. Financial summary
10. Daily transaction limit
11. Calculate interest
12. Logout
Choose an option: Current balance: Rs. 8000.00

--- Customer Dashboard ---
Welcome, Rahul Sharma
Balance: Rs. 8000.00
1. Check balance
2. Deposit money
3. Withdraw money
4. Transfer money
5. Transaction history
6. Mini statement
7. Account details
8. Change PIN
9. Financial summary
10. Daily transaction limit
11. Calculate interest
12. Logout
Choose an option:
--- Mini Statement (Latest 5 Transactions) ---
TXN-2BAC8359 DEPOSIT      Rs. 10000.00  Cash deposit      Rs. 10000.00  17-09-2026 18:23:48
TXN-EEB36097 WITHDRAWAL   Rs. 2000.00  Cash withdrawal   Rs. 8000.00   17-09-2026 18:23:48

--- Customer Dashboard ---
Welcome, Rahul Sharma
Balance: Rs. 8000.00
1. Check balance
2. Deposit money
3. Withdraw money
4. Transfer money
5. Transaction history
6. Mini statement
7. Account details
8. Change PIN
9. Financial summary
10. Daily transaction limit
11. Calculate interest
```

Figure 15.2 — Deposit, withdrawal, mini statement and financial summary output

15.3 Administrator Dashboard and Statistics

```
Smart Banking System - Terminal Output

12. Logout
Choose an option:
--- Financial Summary ---
Cash deposited   : Rs. 10000.00
Cash withdrawn  : Rs. 2000.00
Transfers sent   : Rs. 0.00
Transfers received: Rs. 0.00
Interest earned  : Rs. 0.00
Current balance  : Rs. 8000.00

--- Customer Dashboard ---
Welcome, Rahul Sharma
Balance: Rs. 8000.00
1. Check balance
2. Deposit money
3. Withdraw money
4. Transfer money
5. Transaction history
6. Mini statement
7. Account details
8. Change PIN
9. Financial summary
10. Daily transaction limit
11. Calculate interest
12. Logout
Choose an option: Estimated interest at 4.00%: Rs. 320.00
Credit this interest? (1 = yes, 2 = no):
--- Customer Dashboard ---
Welcome, Rahul Sharma
Balance: Rs. 8000.00
1. Check balance
2. Deposit money
3. Withdraw money
4. Transfer money
5. Transaction history
6. Mini statement
7. Account details
8. Change PIN
9. Financial summary
10. Daily transaction limit
11. Calculate interest
12. Logout
Choose an option: Logged out successfully.

=== SMART BANKING SYSTEM ===
1. Create a new account
2. Customer login
3. Admin login
4. Exit
Choose an option:
--- Admin Login ---
Username: Password: Admin login successful.

--- Admin Dashboard ---
1. View all accounts
2. Search for an account
3. Freeze an account
4. Unfreeze an account
5. Close an account
6. View system statistics
7. Reset daily limits
8. Logout
Choose an option:
--- All Accounts ---
10010001 | Rahul Sharma | SAVINGS | Rs. 8000.00 | ACTIVE

--- Admin Dashboard ---
1. View all accounts
2. Search for an account
3. Freeze an account
4. Unfreeze an account
5. Close an account
6. View system statistics
7. Reset daily limits
8. Logout
Choose an option:
--- System Statistics ---
Total accounts : 1
Savings accounts : 1
Current accounts : 0
Active accounts : 1
Frozen accounts : 0
Closed accounts : 0
Total balance : Rs. 8000.00
Transactions : 2

--- Admin Dashboard ---
1. View all accounts
2. Search for an account
3. Freeze an account
4. Unfreeze an account
5. Close an account
6. View system statistics
7. Reset daily limits
8. Logout
Choose an option: Admin logged out.

=== SMART BANKING SYSTEM ===
1. Create a new account
2. Customer login
3. Admin login
4. Exit
Choose an option: Thank you for using Smart Banking System.
```

Figure 15.3 — Administrator account list and system statistics output